Sistemas de Tiempo Real — Práctica 3

Ejercicio 4: Planificación Round Robin

Ingeniería en Computación

Conjunto de tareas y escala

Se cargan las cuatro tareas del Ejercicio 1 con igual prioridad. Con la escala del banco de medición (1 unidad = 4 ticks = 64 ms):

Tarea	Pin	C (unid.)	T (unid.)	C (ms)	T (ms)
A	PB0	2	12	128	768
B	PB1	2	24	128	1536
C	PB2	2	6	128	384
D	PB3	1	3	64	192

Factor de utilización $U = \frac{2}{12} + \frac{2}{24} + \frac{2}{6} + \frac{1}{3} = 0{,}9167$. Hiperperíodo $\text{lcm}(12, 24, 6, 3) = 24$ unidades = 1536 ms, con 22 unidades de procesador ocupado y 2 de ocio.

Limitación del simulador y cómo se resolvió

En el Ejercicio 3 se midió, instrumentando el propio firmware, que **en Proteus el contador de ticks no avanza mientras una tarea consume CPU**: la columna `dtT` del reporte daba 0 ticks durante una ráfaga de 64 ms. El tick mide tiempo ocioso, no tiempo real.

En el Ejercicio 3 eso se compensaba porque había una sola tarea. Acá no: con $U = 0{,}9167$ solo 2 de cada 24 unidades son ociosas, de modo que el reloj del sistema operativo correría doce veces más lento que el tiempo real. No habría ni cuanto ni activaciones periódicas. Se descartó modificar la biblioteca, de modo que el banco se rediseñó para **no depender del tick**:

- **Reloj.** Un contador propio avanza 1 por cada milisegundo de CPU realmente consumido, dentro del mismo lazo de espera activa calibrado del Ejercicio 3. La tarea Idle consume el tiempo ocioso de la misma forma calibrada, de modo que la suma mide tiempo real.
- **Cuanto.** Lo produce la propia tarea: la ráfaga se parte en trozos de 16 ms y al final de cada uno se llama a `taskYIELD()`, que termina en `vTaskSwitchContext()` y rota la lista de listos igual que lo haría la interrupción del tick.
- **Activaciones.** Las libera un despachador con notificaciones de tarea, llamado al cerrar cada cuanto, de modo que una activación nunca se atrasa más de un cuanto. El reparto se hace con el planificador suspendido para que las cuatro queden liberadas en el mismo instante.

El pin de cada tarea se lleva a alto únicamente mientras tiene el procesador, y a bajo al ceder el cuanto, de manera que el analizador muestra el reparto real y no la ventana de respuesta.

Consecuencia a declarar. Entre prioridades distintas la expropiación sigue siendo la del sistema operativo: la tarea Idle es expropiada por cualquier periódica que se libere. Entre tareas de igual prioridad, en cambio, el reparto es cooperativo y lo fuerza la tarea, no el tick. Es el precio de no modificar la biblioteca.

a) El cuanto de tiempo

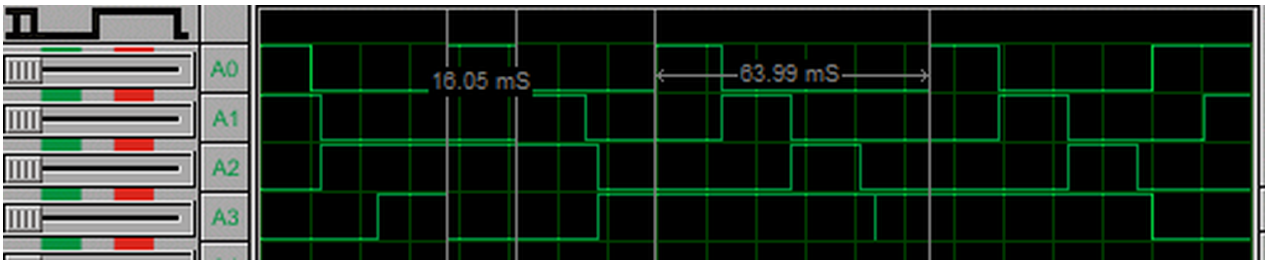


Figura 1: Canales A0–A3 (PB0–PB3 = tareas A–D). Cursores sobre un tramo de ejecución (**16,05 ms**) y sobre una vuelta completa de las cuatro tareas (**63,99 ms**).

El cuanto medido es de **16,05 ms**, contra 16,00 teóricos: error del 0,3 %. La vuelta completa mide **63,99 ms**, exactamente 4×16 ms, lo que confirma que cuando las cuatro tareas están listas el reparto es estrictamente equitativo y cíclico.

¿Coincide con el período del tick? Sí: 16 ms es el valor de `portTICK_PERIOD_MS` en esta biblioteca. Pero por lo explicado más arriba, en este banco el cuanto no lo *produce* la interrupción del tick, sino la tarea al ceder el

procesador. El valor se eligió igual al del tick justamente para que el cronograma sea comparable con el que daría un reparto por expropiación.

b) Fin de la primera ráfaga

Con las cuatro tareas activadas simultáneamente en $t = 0$, el firmware registra el instante en que cada una completa su primera ráfaga (columna `fin1` del reporte):

Tarea	C (ms)	$n \cdot C$ teórico	`fin1` medido	Error
D	64	256	**257**	0,4 %
A	128	512	**508**	0,8 %
B	128	512	**524**	2,3 %
C	128	512	**529**	3,3 %

El modelo es inmediato: con $n = 4$ tareas repartiéndose el procesador en partes iguales, cada una avanza a un cuarto de velocidad y tarda $R \approx n \cdot C$ en completar su ráfaga.

Conviene notar por qué las cuatro compiten durante todo el intervalo. D se reactiva a los 192 ms, cuando todavía no terminó su primera ráfaga (que recién completa a los 257), de modo que **nunca sale de la lista de listos**. El reparto es de a cuatro de punta a punta, y por eso A, B y C tardan $4 \times 128 = 512$ ms y no los 384 que habría dado un reparto de a tres.

c) Tabla de resultados

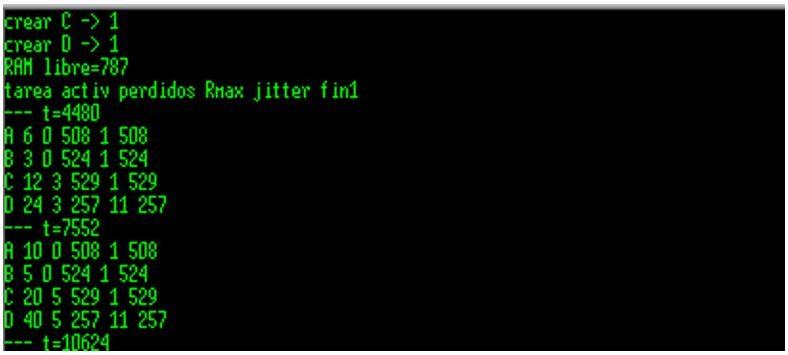


Figura 2: Reporte periódico, un bloque cada 3072 ms (dos hiperperíodos). Columnas: tarea, activaciones, plazos perdidos, $R_{\text{máx}}$, jitter máximo y `fin1`, los cuatro tiempos en ms.

Tarea	Activaciones	Plazos perdidos	$R_{\text{máx}}$ (ms)	Jitter (ms)	T (ms)
A	10	0	508	1	768
B	5	0	524	1	1536
C	20	5	529	1	384
D	40	5	257	11	192

Las activaciones crecen entre reportes en +4, +2, +8 y +16, que es exactamente lo que corresponde a dos hiperperíodos: A se activa 2 veces por hiperperíodo, B 1, C 4 y D 8. Esa comprobación valida el reloj propio: si midiera mal, los cuatro incrementos se desviarían.

Dos observaciones sobre la tabla. Primero, $R_{\text{máx}}$ coincide exactamente con `fin1` en las cuatro tareas: el peor tiempo de respuesta de toda la corrida ocurre en la primera ráfaga, con las cuatro tareas liberadas juntas. Es el *instante crítico* del análisis clásico de planificabilidad, observado experimentalmente. Segundo, el jitter de 11 ms de D es consecuencia de que el despacho ocurre al cerrar cada cuanto: D se activa ocho veces por hiperperíodo, de modo que es la que más veces cae en medio de un cuanto.

d) Por qué se pierden plazos con $U < 1$

Porque $U < 1$ garantiza únicamente que el trabajo *entra* en el procesador; no dice nada sobre *cuándo* entra. Round Robin reparte el procesador en partes iguales e ignora por completo los plazos.

Con n tareas listas, cada una recibe una porción $1/n$ del procesador. Una tarea cumple su plazo si le alcanza con esa porción, es decir si

$$n \cdot C_i \le T_i \qquad \Longleftrightarrow \qquad \frac{C_i}{T_i} \le \frac{1}{n}$$

Tarea	C_i/T_i	Porción recibida	$n \cdot C_i$	T_i	Resultado
A	16,7 %	25 %	512	768	cumple
B	8,3 %	25 %	512	1536	cumple
C	33,3 %	25 %	512	384	**pierde**
D	33,3 %	25 %	256	192	**pierde**

La porción equitativa es $1/4 = 25$ %. Entonces:

A y B piden menos que su porción equitativa y les sobra margen. C y D piden un tercio del procesador y reciben un cuarto: les falta trabajo por hacer cuando vence el plazo. La utilización total sigue siendo 0,9167 y el sistema no está sobrecargado; el problema es de reparto, no de capacidad.

e) Tarea más perjudicada

C y D son las dos perjudicadas, porque son las únicas que piden más que su porción equitativa. Entre ambas, los datos marcan a **C**.

Las dos piden lo mismo (1/3) y las dos pierden un plazo por hiperperíodo, pero C se activa 4 veces por hiperperíodo y D se activa 8. En proporción, C pierde el **25 %** de sus activaciones y D el **12,5 %**. El exceso sobre el plazo también es algo mayor en C: $529/384 = 1{,}38$ contra $257/192 = 1{,}34$.

La razón de fondo es que C necesita el doble de cómputo por activación (128 ms contra 64) con solo el doble de plazo, de modo que bajo reparto equitativo acumula su déficit en la mitad de oportunidades. D, al activarse más seguido, reparte la misma pérdida absoluta entre el doble de trabajos.

El resultado general es el que motiva las políticas basadas en plazos: el reparto equitativo castiga justamente a las tareas más exigentes en tiempo, y trata igual a C, que tiene 384 ms para hacer 128 de cómputo, que a B, que tiene 1536 ms para hacer lo mismo.